Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Tasarım nedir? (ekrandan, 00:20:00)
  - Çözümü oluşturma sürecinin ilk adımı analizdir (problemi anlama). İkinci adımı tasarımdır: belirli bir mimariye uygun olacak şekilde düşünülen yazılım uzayı modeli.
  - Tasarım, kararlar vermektir. Kararlar genellikle işlevsel olmayan kriterlerden (performans, kullanıcı sayısı, tepki süresi vb.) hangisinden ödün verileceğiyle ilgilidir; hepsi aynı anda karşılanamaz.
  - Müşteri, son kullanıcı, teknik beklentiler ve rakip ürünler bu kararları etkiler.
  - Bazen daha ayrıntılı analiz gerekir: simülasyonlar (ör. yük testi gibi 1000 ya da 10000 kullanıcıyla tepkileri ölçmek), prototipler (birden fazla çözümü küçük ölçekte denemek) ve tasarım çalışmaları.
- Tasarım çalışmaları (design studies)
  - Kavram bina mimarisinden miras alınmıştır. Mimar, eğimli bir arazi gibi bir problem uzayında farklı çözüm yaklaşımlarını ölçekli modellerle dener. Aynı yaklaşım araba, uçak ve giysi tasarımında da kullanılır.
  - Tasarımı etkileyen faktörler titiz ve sistematik biçimde değerlendirilir: önce başarı kriterleri belirlenir, sonra ölçüm yapılır, hangi sonuçların tatmin edici olduğu belirlenir ve en uygun yaklaşımla devam edilir. Örnek kriter: yağmurlu bir bölgede birim zamanda tahliye edilebilen su miktarı.
  - Yazılımda kavramsal model alınır, en az iki farklı çözüm (farklı mimari, olay tabanlı yapı vb.) denenir. Bu ek emek ve zaman ister.
  - Binadaki maliyet, malzemenin bulunabilirliği, yasalar ve çevre faktörlerinin yazılımdaki karşılıkları: maliyet (uygulanabilirlik), yeniden kullanılabilir bileşenler ve kütüphanelere erişim (lisans), lisans kısıtları, aynı sunucudaki diğer hizmetlere etkisi (bant genişliği, sistem kaynakları), performans, bellek, üretim zamanı.
- Tipik bir tasarım çalışması raporu (ekrandan, 00:30:00)
  - 1 Bağlam: geçmiş, motivasyon, anahtar kelimeler ve tanımlar.
  - 2 Araştırma soruları: bağımlı değişkenler cinsinden karşılaştırılan işlevsel olmayan gereksinimler ve etkilenen faktörler (ör. ön işleme hesapları değişince zaman ve bellek kullanımı nasıl değişiyor?).
  - 3 Konuların tanımları: çözüm yaklaşımları.
  - 4 Deneye özel koşul ve durumlar: konfigürasyon özellikleri (tekrar edilebilirlik için).
  - 5 Değişkenler: değiştirilen durumlar bağımsız, elde edilen sonuçlar bağımlı değişkenlerdir; adı, tanımı ve ölçü birimi verilir.
  - 6 Metot, 7 Deney sonuçları, 8 Tartışma, 9 Sonuç (araştırma sorularına açık cevap).
  - Özellikle kontrata dayalı, kritik (ör. askerî) projelerde bu ayrıntıda yapılması gerekir.
- Kütüphane örneği: gereksinimlerden kavramsal modele (ekrandan, 00:40:00)
  - Gereksinimler: her ziyaretçinin tekil bir kütüphane kartı vardır; kütüphane ad, adres, telefon ve kart numarasını bilir; ziyaretçinin ödünç aldığı parçalar, teslim zamanları ve ödemediği gecikme cezaları her an bilinebilir ya da hesaplanabilir; 12 yaş altı çocuklar en fazla 5 parça ödünç alabilir; kitap ve DVD ödünç alınabilir; kitaplar 3 hafta, çok satan kitaplar 2 hafta, DVD'ler 2 hafta için verilir; gecikme cezası günlük 10 TL'dir ve parçanın fiyatını geçemez; başvuru kitapları ve dergiler ödünç alınamaz; kütüphanede olmayan kitap ya da DVD talep edilebilir; her parça için bir kez süre uzatılabilir, parça için çok talep varsa geri getirilmelidir.
  - İsimler aday sınıf (kırmızı), özellik olabilecekler yeşil işaretlenir.

  - İlk sınıflar (ekrandan, 00:44:10): KütüphaneKartı (KKartNo), Ziyaretçi (ad, adres, telefon), Ceza, Parça (fiyat), Çocuk (yaş), BaşvuruKitabı, Dergi, Kitap, DVD, ÇokSatan, Para, Tarih.
  - Eleme: KütüphaneKartı yalnız numara tutar ve ziyaretçiyle birebir ilişkilidir, Ziyaretçi'nin özelliği olur. Çocuk yalnız yaşıyla ayrılan bir ziyaretçidir, yaş Ziyaretçi'ye taşınır. ÇokSatan, Kitap'ın Boolean özelliği olur.
  - Davranışlar (ekrandan, 00:50:00): Ziyaretçi'ye ödünçAldıkları, teslimTarihleri, ödenmemişCezalar, ödünçAl, talepEt.
  - Genelleştirme (ekrandan, 00:51:30): Parça, ÖdünçVerilemezParça (BaşvuruKitabı, Dergi) ve ÖdünçVerilebilirParça (Kitap, DVD) olarak ayrılır; ödünç alma ilişkisi yalnız ÖdünçVerilebilirParça'ya bağlanır, böylece dergi ve başvuru kitabının ödünç alınamayacağı modelin yapısından çıkar.
  - Talep etme Başlık (title) sınıfına bağlanır: talep belirli bir kopyaya değil başlığa yapılır; Başlık, parçaların birleşmesidir (1'e çok).
  - Ödünç ilişki sınıfı (ekrandan, 00:57:10): teslim tarihi, uzatma sayısı, ödünç durumu ve ceza ne kitabın ne ziyaretçinin özelliğidir; ödünç alma ilişkisinin özelliğidir. Fiyat parçada kalır.
  - Son model (ekrandan, 01:00:00): Başlık (adı, yazar, barkod: String); Ziyaretçi (ad, adres, telefon, KKartNo: String; doğumTarihi: Tarih; /yaş: Integer); Ödünç (uzatıldıMı: Boolean = false; teslimTarihi: Tarih; /ceza: Para; neZamanGeldi: Tarih); ÖdünçVerilebilirParça (günBaşınaCeza: Para = 10; ödünçPeriyodu: Tarih; fiyat: Para); Kitap (çokSatan: Boolean); `<<utility>>` Para, Tarih ve İşletimSistemi (`tarihVer(): Tarih`). Başlık ile Ziyaretçi arasında çoka çok `talepEder`.
- Biçimsel belirtim (formal specification)
  - Belirtim, programın ne yapacağının tanımıdır. Müşteriden, son kullanıcıdan ya da analiz takımından gelebilir; iki satırlık gayriresmî bir talep, yapılandırılmış bir belge ya da matematiksel ifade olabilir.
  - Biçimsel belirtim, matematik notasyonuyla herkesin aynı şekilde anlayacağı kesin ifadeler kurmaktır. İki araç vardır: birinci derece mantık (FOL) ve ona dönüştürülebilen, daha okunaklı, sözde koda yakın OCL.
- Önerme mantığı (ekrandan, 01:30:00)
  - Önerme, doğru ya da yanlış olan ifadedir. Komut ("kalemi eline al"), soru ve anlamsız ifadeler önerme değildir.
  - Bağlaçlar: $\neg p$ (değil), $p \wedge q$ (ve), $p \vee q$ (kapsayan veya), $p \rightarrow q$ (çıkarım; $\equiv \neg(p \wedge \neg q)$), $p \leftrightarrow q$ (iki koşullu; $p \rightarrow q$ ve $q \rightarrow p$), $\top$ doğru, $\bot$ yanlış.
  - Doğruluk tablosu (ekrandan, 01:32:00) ve operatör öncelikleri (ekrandan, 01:33:20): $\neg \ \wedge \ \vee \ \rightarrow \leftrightarrow$; $\neg$ takip eden değişkene yapışır; $\wedge$ ve $\vee$, $\rightarrow$'dan daha sıkı bağlıdır.
  - Örnek: A "bu akşam uyanık olacağım", B "bu akşam ay tutulmasını göreceğim", C "bu akşam ay tutulması var". "Uyanık olmazsam ay tutulmasını göremem": $\neg A \rightarrow \neg B$. "Uyanığım ama ay tutulması yoksa göremem": $A \wedge \neg C \rightarrow \neg B$.
  - De Morgan (ekrandan, 01:36:30): $\neg(p \wedge q) \equiv \neg p \vee \neg q$, $\neg(p \vee q) \equiv \neg p \wedge \neg q$; $p \rightarrow q \equiv \neg(p \wedge \neg q)$, $\neg(p \rightarrow q) \equiv p \wedge \neg q$, $p \rightarrow q \equiv \neg q \rightarrow \neg p$.
- Birinci derece mantık (FOL) (ekrandan, 01:40:00)
  - Önerme bağlaçlarını nesnelerin özellikleri hakkında önerme kurmak için kullanır: ifadeler (predicates) özellik tanımlar ve doğru/yanlış üretir; fonksiyonlar nesneleri nesnelere eşler; niceleyiciler çoklu nesneler için önerme kurar.
  - Örnek: $Sever(Sen, Yumurta) \wedge Sever(Sen, Domates) \rightarrow Sever(Sen, Menemen)$. Sen, yumurta, domates, menemen sabittir (nesne); sever ifadedir.
  - Eşitlik ($=$) yalnız nesnelere uygulanır; önermeler için $\leftrightarrow$ kullanılır.
  - Fonksiyon nesne döndürür (ör. birinin en sevdiği filmin yıldızı). Nesnelere bağlaç uygulanamaz ("Venüs → Güneş" yanlıştır); fonksiyonlara önerme argüman olarak verilemez.
- Niceleyiciler (ekrandan, 01:46:00)
  - Evrensel $\forall$: tüm değişkenler için doğrudur. $\forall n.(n \in \mathbb{N} \rightarrow (Even(n) \leftrightarrow Even(n^2)))$.
  - Varoluşsal $\exists$: bazı değişkenler için doğrudur. $\exists x.(Even(x) \wedge Prime(x))$ (tek örnek 2), $\exists x.(TallerThan(x, me) \wedge LighterThan(x, me))$.
  - Kapsam (ekrandan, 01:50:00): değişken yalnız nicelenen ifade içinde geçerlidir. $(\forall x.Loves(You, x)) \rightarrow (\forall y.Loves(y, You))$: "herkesi seversen herkes de seni sever". Aynı adı iki kez kullanmak doğrudur ama okunaklılığı azaltır.
  - Yanlış parantezleme: $(\forall x.P(x)) \vee R(x) \rightarrow Q(x)$; doğrusu $\forall x.(P(x) \vee R(x) \rightarrow Q(x))$.

- İfadeleri kurgulamak (ekrandan, 01:54:40; 02:00:00)
  - "All P's are Q's": $\forall x.(P(x) \to Q(x))$. $\forall x.(Puppy(x) \land Cute(x))$ yanlıştır.
  - "Some P is a Q": $\exists x.(P(x) \land Q(x))$. $\exists x.(Blobfish(x) \to Cute(x))$ yanlıştır.
  - "Everybody loves someone else": $\forall p.(Person(p) \to \exists q.(Person(q) \land p \neq q \land Loves(p,q)))$.
  - "There is a person that everyone else loves": $\exists p.(Person(p) \land \forall q.(Person(q) \land p \neq q \to Loves(q,p)))$.
- Niceleyici sırası ve değilleme (ekrandan, 02:05:00; 02:10:00)
  - $\forall x.\exists y.P(x,y)$: her x için P'yi doğru yapan bir y vardır. $\exists x.\forall y.P(x,y)$: öyle bir x vardır ki her y için P doğrudur.
  - Değilleme: $\neg$ içeri itilir, niceleyici değiştirilir. $\neg\forall x.P(x) \equiv \exists x.\neg P(x)$, $\neg\exists x.P(x) \equiv \forall x.\neg P(x)$.
  - $\neg\forall x.\exists y.Loves(x,y) \equiv \exists x.\forall y.\neg Loves(x,y)$: "kimseyi sevmeyen biri vardır".
- Küme işlemleri (ekrandan, 02:11:30; 02:14:40)
  - $Set(S)$: S bir kümedir; $x \in y$: x, y'nin elemanıdır.
  - Boş küme vardır: $\exists S.(Set(S) \land \neg\exists x.x \in S)$ ya da $\exists S.(Set(S) \land \forall x.\neg(x \in S))$.
  - İki küme ancak aynı elemanları içeriyorsa eşittir: $\forall S.(Set(S) \to \forall T.(Set(T) \to (S = T \leftrightarrow \forall x.(x \in S \leftrightarrow x \in T))))$.
  - Kümeyle sınırlı niceleyici: $\forall x \in S.P(x)$, $\exists x \in S.P(x)$. Ör. 12 yaş altı çocukların 5 parça sınırı kümelenerek ifade edilebilir.
  - Sonraki adım OCL'dir: FOL'a dönüştürülebilir, sözde kod gibi okunur ve UML diyagramlarına kısıt eklemeyi sağlar.

Hocanın Özellikle Vurguladığı Kısımlar

- Tasarım ödün vermektir
  - Bütün işlevsel olmayan gereksinimler aynı anda karşılanamaz; tasarım, bunlardan hangisinden ne kadar ödün verileceğine dair kararların kümesidir.
- En az iki çözüm deneyin
  - Tasarım çalışmasında en az iki farklı yaklaşımın denenmesi mutlaka tavsiye edilir. Bu zaman kaybı gibi görünür, ama en iyi çözüme ancak denemelerle ulaşılır.
- Bütün adayları önce ortaya koyun
  - Tanınmayan bir problem uzayında önce bütün olası sınıf adayları çıkarılmalı, sonra elenmelidir; bu, bir şeyi kaçırma olasılığını azaltır. Bu iş ekip hâlinde tartışılarak yapılırsa daha başarılı olur.
- Değişebilecek bir durumu alt sınıf yapmayın
  - Çocuk 13 yaşına girince ya da çok satan kitap rağbetten düşünce nesnenin sınıfı değişmez; nesneyi öldürüp değerlerini yeni sınıftan bir nesneye kopyalamak saçma olur. Esas sınıflar değişmez, tutarlı yapılar olmalıdır; değişen durum özellik olarak tutulur.
- Değişime açık noktaları parametrik yapın
  - Para birimi (TL, dolar, euro), ceza politikası, çocuk sınırı gibi değişebilecek şeyler ayrı tutulmalı ya da parametrik yapılmalıdır.
- Özelliği ait olduğu yere verin
  - Teslim tarihi, uzatma sayısı ve ceza kitabın değil ödünç ilişkisinin özelliğidir. Ceza ödünç içinde tutulursa ziyaretçinin kitabı bırakıp cezayı ödememesi gibi ayrı yaşayan ceza nesneleri oluşmaz.
- Talep kopyaya değil başlığa yapılır
  - Bir kitabın beş kopyası varsa kullanıcı "ikinci kopyayı istiyorum" diyemez; talep başlığa yapılmalıdır.
- "All P's are Q's" ve "Some P is a Q" kalıpları
  - Hoca bunları ezber gibi kullanılabilecek standart kalıplar olarak verdi: evrensel niceleyiciyle $\to$, varoluşsal niceleyiciyle $\land$.
- Parantezleme ve kapsam
  - Niceleyicinin kapsamı bütün ifadeyi kaplamalıdır; yarısında tanımlı, yarısında tanımsız değişken yanlış ifade üretir.

Kısa Tekrar Notları

- Analiz = problemi anlama; tasarım = mimariye uygun yazılım uzayı modeli.
- Tasarım = kararlar = işlevsel olmayan gereksinimler arasında ödün.
- Ayrıntılı analiz araçları: simülasyon, prototip, tasarım çalışması.
- Tasarım çalışması raporu: bağlam, araştırma soruları, konular, deney koşulları, değişkenler, metot, sonuçlar, tartışma, sonuç.
- Kütüphane modeli: Ziyaretçi, Başlık, Parça, ÖdünçVerilemezParça (BaşvuruKitabı, Dergi), ÖdünçVerilebilirParça (Kitap, DVD), Ödünç ilişki sınıfı, Para/Tarih/İşletimSistemi utility.
- Çocuk ve ÇokSatan sınıf değil, özelliktir (yaş, çokSatan).
- Gecikme cezası: günlük 10 TL, en fazla parçanın fiyatı.
- Önerme: doğru ya da yanlış ifade. Komut ve soru önerme değildir.
- $p \to q \equiv \neg(p \wedge \neg q) \equiv \neg q \to \neg p$.
- De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
- Öncelik: $\neg$, $\wedge$, $\vee$, $\to$, $\leftrightarrow$.
- FOL: ifade (doğru/yanlış), fonksiyon (nesne döndürür), niceleyici ($\forall$, $\exists$).
- $=$ yalnız nesneler için; $\leftrightarrow$ önermeler için.
- $\forall x.(P(x) \to Q(x))$; $\exists x.(P(x) \wedge Q(x))$.
- Değilleme: $\neg$ içeri, $\forall \leftrightarrow \exists$ yer değiştirir.
- Boş küme: $\exists S.(Set(S) \wedge \forall x.\neg(x \in S))$.

Detaylı Açıklamalar

Dersin ilk bölümünde tasarımın ne olduğu ele alınır. Hoca daha önceki derslerde anlatılan iki aşamalı süreci hatırlatır: önce problem anlaşılır (analiz), sonra çözüm oluşturulur (tasarım). Tasarım, analizdeki kavramsal modeli belirli bir mimariye uygun bir yazılım modeline dönüştürür. Bu dönüşümde verilen kararlar, işlevsel olmayan gereksinimlerden hangisinden ödün verileceğine dair kararlardır. Kim kime nasıl servis verecek, yazılım sınıfları, dağıtım planı ve bileşen yapıları nasıl olacak gibi sorular performans, kullanıcı sayısı ve tepki süresi gibi kısıtlara göre farklı cevaplanır. Bazen bu kararlar için simülasyon (yük testi gibi), prototip ya da tasarım çalışması gerekir.

Tasarım çalışması kavramı bina mimarisinden gelir. Mimar eğimli bir arazide farklı çözümleri ölçekli modellerle dener, kriterlerini (ör. yağmur suyunun tahliyesi) belirler, ölçer ve en iyi sonucu veren tasarımla devam eder. Yazılımda da aynı problem uzayında en az iki farklı mimari ya da kalıp denenir. Bina tarafındaki maliyet, malzeme, yasa ve çevre faktörleri yazılımda maliyet, yeniden kullanılabilir bileşenler ve kütüphanelerin lisansı, paylaşılan sunucudaki diğer hizmetlere etki, performans ve bellek olarak karşımıza çıkar. Çalışmanın sonunda dokuz bölümlü bir rapor yazılır. Bu ayrıntı her projede beklenmez, ama kritik ve kontrata dayalı işlerde gerekir.

Ardından hoca, analizden tasarıma geçişi hatırlatmak için bir kütüphane örneği çözer. Kendi başına küçük bir kütüphane işleten bir kişi, arkadaşına sistemini anlatır; arkadaşı ona bir bilgi sistemi yapmayı önerir. Gereksinim metnindeki isimler aday sınıf ya da özellik olarak işaretlenir. İlk listede kütüphane kartı, ziyaretçi, ceza, parça, çocuk, başvuru kitabı, dergi, kitap, DVD, çok satan, para ve tarih vardır. Sonra eleme yapılır: yalnız bir numara tutan ve ziyaretçiyle birebir ilişkili kart bir özelliğe dönüşür; çocuk ve çok satan, nesnenin zamanla değişebilen durumları oldukları için alt sınıf değil özellik olur. Ziyaretçiye ödünç aldıklarını, teslim tarihlerini ve ödenmemiş cezaları sorgulama, ödünç alma ve talep etme davranışları eklenir.

İlişkiler kurulurken önce parçalar ödünç verilebilir ve verilemez diye genelleştirilir. Ödünç alma ilişkisi yalnız ödünç verilebilir parçalara bağlandığında, dergi ve başvuru kitabının ödünç alınamayacağı ayrı bir kontrol yazmadan modelin yapısıyla gösterilmiş olur. Talep ise ilk önce parçaya bağlanır; ama bu, bütün kopyaların birden talep edilmesi ya da belirli bir kopyanın seçilmesi gibi gerçek hayatta olmayan durumlara yol açar. Bu yüzden talep, parçaların birleştiği Başlık sınıfına bağlanır. Son olarak teslim tarihi, uzatma sayısı, ödünç durumu ve ceza Ödünç adlı bir ilişki sınıfına taşınır. Modele tipler, ilk değerler (günlük ceza 10), türetilmiş özellikler (yaş, ceza), çoğullamalar ve `<<utility>>` stereotipleri eklenir. Hoca, sonraki adımın bu kavramların hangi yazılım sınıflarına, servislere, veri depolarına ve mesaj kuyruklarına dönüşeceğine karar vermek olduğunu; bunun da tasarım çalışmalarıyla, ör. “1 milyon kullanıcıya

10 saniyenin altında hizmet” gibi bir gereksinimi karşılayacak şekilde yapılacağını söyler.

İkinci bölümde biçimsel belirtime geçilir. Belirtim programın ne yapacağını tanımlar; biçimsel belirtim bunu matematiksel ve tek anlamlı ifadelerle yapar. Önce önerme mantığı hatırlatılır: bağlaçlar, doğruluk tablosu, operatör öncelikleri ve De Morgan kuralları. Hoca De Morgan dönüşümleriyle bir belirtimin daha sade ya da daha hızlı çalışacak denk bir ifadeye dönüştürülebileceğini söyler.

> **Ders sonrası not:** Slaytta `if (!(p() && q()))` yerine `if (!p() || !q())` yazmanın “daha hızlı” olacağı söylenir. Kısa devre değerlendirmesi olan dillerde iki biçim de aynı fonksiyonları aynı sırada çağırır (p() yanlışsa q() hiç çağrılmaz); büyük olasılıkla belirgin bir hız farkı yoktur. De Morgan’ın asıl yararı ifadeyi sadeleştirmek ve denk biçimler bulmaktır.

Birinci derece mantık, önerme mantığına nesneler, ifadeler (predicate), fonksiyonlar ve niceleyiciler ekler. İfadeler doğru ya da yanlış, fonksiyonlar nesne döndürür; bu ikisi karıştırılmamalıdır. Evrensel ve varoluşsal niceleyicilerle “bütün” ve “bazı” ifadeleri kurulur. Hoca iki standart kalıbın ezber gibi kullanılmasını önerir: “Bütün P’ler Q’dur” için $\forall x.(P(x) \rightarrow Q(x))$, “Bazı P’ler Q’dur” için $\exists x.(P(x) \wedge Q(x))$. Bağlaçların yanlış seçilmesi İngilizcesi doğru görünen ama anlamı yanlış ifadeler üretir. Daha karmaşık cümleler (herkes başka birini sever; herkesin sevdiği biri vardır) bu kalıpların birleştirilmesiyle kurulur. Niceleyicilerin sırası anlamı değiştirir. Değillemede $\neg$ içeri itilir ve niceleyici değiştirilir; “herkes birini sever” cümlesinin değili “kimseyi sevmeyen biri vardır” olur. Son olarak boş kümenin varlığı ve küme eşitliği FOL ile yazılır, niceleyicilerin bir kümeyle sınırlandırılabileceği gösterilir. Ders, bir sonraki derste aynı belirtimlerin OCL ile yazılacağı söylenerek kapanır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.